CRYPTO LIQUIDITY
AI PREDICTION SYSTEM

Deep Technical Report & Step-by-Step Build Guide

A real-time ML system that reads Coinglass liquidity heatmaps, auto-interacts with the UI, learns long/short liquidity zones, and predicts short-term crypto price movement. Built for both PC and Mobile.

Version 1.0 | March 2026 | Project: LiquidityAI

1. What Is This Project? (Plain English)

Think of this system as a smart trading assistant that watches your screen. Instead of you manually reading the Coinglass heatmap, clicking around, and guessing where price will go, this AI does it all automatically.

Here is the simple idea: On the Coinglass Liquidity Heatmap, bright/dark colors show where massive buy or sell orders are sitting. When price moves towards those zones, it often gets attracted to them (like a magnet). This AI learns those patterns and predicts the next move.

The 5 things this system does:

1. Watches your screen in real time and reads the heatmap.
2. Automatically clicks and adjusts the threshold slider bar for best color visibility.
3. Clicks on the map at every price level to read how much liquidity is there.
4. Learns which zones are long (buy) clusters and which are short (sell) clusters.
5. Predicts where price is most likely to move next and logs everything to get smarter over time.

2. High-Level System Architecture

Below is the full journey of data through the system, from your screen to a prediction output:

[Your Screen / Mobile] --> Screen Capture --> OCR + Vision Module --> Feature Extractor --> Threshold Learner --> Liquidity Analyzer --> Online ML Model --> Prediction Engine --> Log System --> Visual Overlay

Component	What It Does
Screen Capture	Takes a live snapshot of your screen every 1-5 seconds
OCR + Vision Module	Reads numbers, colors, and layout from the heatmap image
Feature Extractor	Pulls out useful numbers: liquidity per level, clusters, sweeps
Threshold Learner	Decides the minimum liquidity that matters (auto-adjusts the slider)
Liquidity Analyzer	Finds where buyers and sellers are concentrated
Online ML Model	Learns and predicts price direction in real time

Prediction Engine	Outputs UP / DOWN / NEUTRAL with a confidence score
Log System	Records every prediction, real outcome, and heatmap state
Visual Overlay	Draws lime-colored predicted path on top of the chart

3. ML Concepts Used (Explained Simply)

This project combines several machine learning techniques. Here is each one explained so that even a beginner can understand it.

3.1 Computer Vision (Seeing the Screen)

The system needs to see your screen the same way you do. It uses Computer Vision, which is teaching a computer to understand images.

- Screenshot capture: Takes a photo of your screen every few seconds.
- Color detection: Finds the bright red/dark purple zones that mean high liquidity.
- OCR (Optical Character Recognition): Reads the numbers on the heatmap (like 1.4M, 2.0M).
- Template matching: Finds the slider bar, buttons, and price axis positions automatically.

Libraries used: OpenCV (Python), Pillow, Tesseract OCR.

3.2 Feature Engineering (Turning Pixels into Numbers)

Raw images are not useful for ML directly. We convert what we see into a set of numbers called features. Think of features as measurements that describe the current state of the market.

Feature Name	What It Measures	Why It Matters
upper_liquidity	Total liquidity above current price	High = price likely to go up to fill it
lower_liquidity	Total liquidity below current price	High = price likely to drop to fill it
liquidity_diff	Upper minus lower liquidity	Positive = more sellers, Negative = more buyers
cluster_count_above	Number of strong zones above price	More clusters = stronger resistance
cluster_count_below	Number of strong zones below price	More clusters = stronger support

recent_sweeps	How many zones disappeared recently	Sweeps = momentum signal
nearest_long_dist	Distance to nearest buy cluster	Closer = sooner price reaction
nearest_short_dist	Distance to nearest sell cluster	Closer = sooner price resistance
price_velocity	How fast price is moving right now	Fast + cluster near = high probability move

3.3 Online Learning (Gets Smarter Over Time)

Normal machine learning trains on past data, then runs. This system uses Online Learning, which means it keeps learning while it is running. Every new tick of data, the model updates itself.

Why is this important? Crypto markets change every minute. A model trained on last week's data might be wrong today. With online learning, the model adapts as conditions change.

- Algorithm: Incremental Random Forest or SGD Classifier (Stochastic Gradient Descent).
- Every prediction is compared to the real outcome.
- The model weights are updated to be more accurate next time.

3.4 Anomaly Detection (Finding Unusual Liquidity)

The system watches for unusual spikes in liquidity, called anomalies. These are moments when a huge wall of buy or sell orders appears or disappears suddenly.

- Method: Z-score or Isolation Forest algorithm.
- If a zone suddenly has 10x more liquidity than normal, this is flagged as a high-signal event.
- These events are often followed by fast, predictable price moves.

3.5 Adaptive Threshold Learning

The slider bar at the top of the Coinglass heatmap controls what minimum liquidity level gets colored. The system learns to set this threshold automatically for the best visibility and signal quality.

```
threshold = mean(all_liquidity_levels) + k * std(all_liquidity_levels)
```

K starts at 1.5 and adjusts based on how well predictions perform. If too many false signals, K goes up. If missing real signals, K goes down.

3.6 Time-Series Pattern Recognition

The system also looks at sequences of liquidity states, not just the current snapshot. This is time-series analysis.

- It keeps a rolling window of the last 20 ticks.
- It looks for patterns like: liquidity builds slowly then disappears fast = sweep incoming.
- This adds context to the prediction beyond just the current moment.

3.7 Log-Based Reinforcement (Learning From Mistakes)

Every prediction is logged. After each trade or move, the system checks: Was I right? If yes, the confidence score for that pattern goes up. If no, it goes down. Over hundreds of predictions, the system builds a memory of what works for each pair.

---

4. How the Automation Works (Auto-Clicking)

The system uses a technique called Robotic Process Automation (RPA) to interact with the Coinglass app just like a human would, but faster and more precisely.

4.1 Finding UI Elements

Before clicking anything, the system must know where things are on screen. It does this by:

- Taking a screenshot at startup.
- Using template matching to find the slider bar, heatmap area, and price axis.
- Saving the coordinates of each element as reference points.

4.2 Auto-Adjusting the Threshold Slider

The slider bar is the key control. Here is exactly how the system adjusts it:

6. Takes a screenshot and scans the heatmap colors.
7. Counts how many cells are colored vs empty. Ideal is around 30-40% colored.
8. If too many cells colored (noisy): moves slider right to increase threshold.
9. If too few cells colored (missing data): moves slider left to decrease threshold.
10. Uses pyautogui to click and drag the slider to the calculated position.
11. Repeats every 30 seconds or when the pair changes.

4.3 Reading Liquidity at Each Price Level

To read exact liquidity values, the system either reads numbers via OCR from visible text, or uses the tooltip method:

12. Moves mouse to each price row on the heatmap.
13. Waits for the tooltip popup to appear.
14. Captures and reads the tooltip number with OCR.
15. Stores the value: {price_level: value}.
16. Repeats for every visible price row (usually 15-30 rows).

This process takes about 3-8 seconds per scan. After scanning, the system builds a full picture of the current liquidity distribution.

---

5. The Prediction Engine (Step by Step)

Here is exactly how the system goes from raw data to a prediction:

Step 1: Build feature vector from current scan

```
features = [upper_liq, lower_liq, diff, clusters_above, clusters_below, sweeps,
price_velocity, nearest_long, nearest_short]
```

Step 2: Normalize features

All numbers are scaled to 0-1 range using the min/max values seen so far. This prevents large numbers from dominating small ones.

Step 3: Run the ML model

```
probabilities = model.predict_proba(features)  # Returns [P(UP), P(DOWN),
P(NEUTRAL)]
```

Step 4: Apply confidence threshold

```
if P(UP) > 0.60: predict UP elif P(DOWN) > 0.60: predict DOWN else: predict NEUTRAL
(no trade signal)
```

Step 5: Output prediction with target

```
{ direction: UP, confidence: 0.78, target_price: 1.3720, reason: 'Strong buy
cluster at 1.372, distance 0.003' }
```

Step 6: Log everything

```
log = { timestamp, pair, features, prediction, actual_outcome, correct: True/False
}
```

---

6. Logging System (The Brain Memory)

The logs are what make the system smarter over time. Every single prediction, scan, and outcome is saved. Here is what the log structure looks like:

```
logs/   xrpusdt_predictions.csv     <- All predictions for XRPUSDT
xrpusdt_heatmap_states.npy  <- Raw feature arrays per tick   model_weights.pkl          <-
Saved model (updated live)   performance_stats.json      <- Accuracy, win rate per direction
anomaly_events.csv          <- Flagged unusual liquidity events
```

Over time, the system learns pair-specific behavior. XRPUSDT may behave differently to BTCUSDT when liquidity builds at a certain level. The logs capture these differences automatically.

---

7. How It Works on PC vs Mobile

PC Version

On PC, the system runs as a Python background process. You open Coinglass in your browser, and the AI watches your screen.

PC Feature	Implementation
Screen capture	mss or pyautogui.screenshot() - very fast
Mouse control	pyautogui.moveTo(), click(), drag()
OCR	Tesseract via pytesseract
Visual overlay	A transparent PyQt5 window on top of browser
Performance	~1-3 second update cycle, very smooth
Setup effort	Medium - install Python + libraries + run script

Mobile Version (Android)

On mobile, it is slightly more complex because you cannot run Python directly. There are two approaches:

Option A: Remote Server Approach

Run the Python AI on your PC or a cloud server. The mobile phone streams its screen to the server. The AI processes it and sends back predictions to display on your phone.

- Tools: scrcpy for screen mirroring, socket for data transfer.
- Phone shows predictions via a simple web app opened in browser.
- No apps to install on phone.

Option B: Android Automation Approach

Use ADB (Android Debug Bridge) to control your Android phone from PC. The AI can tap the screen, adjust the slider, and read data remotely.

```
adb shell input tap 350 870         # tap on slider adb shell input swipe 300 870
450 870  # drag slider right adb exec-out screencap -p > frame.png  # get
screenshot
```

- Works without rooting your phone.
- Requires phone connected to PC via USB or WiFi ADB.
- AI runs on PC, phone is just the display and input device.

Aspect	PC vs Mobile Comparison
Screen capture speed	PC: ~100ms | Mobile ADB: ~500ms
Click accuracy	PC: pixel-perfect | Mobile: ~5-10px variance
Setup complexity	PC: easier | Mobile: needs ADB setup
Can run standalone?	PC: yes | Mobile: needs PC assistance
Best for live trading?	PC recommended for lowest latency

8. Step-by-Step Build Guide for PC

Step 1: Install Required Tools

```
# Python 3.10+ required pip install opencv-python pillow pytesseract pyautogui mss
pip install scikit-learn numpy pandas matplotlib pyqt5 pip install river  # for
online learning
```

Also install Tesseract OCR from: https://github.com/UB-Mannheim/tesseract/wiki

Step 2: Calibrate Your Screen

Run the calibration script once. It will open Coinglass, take a screenshot, and ask you to click on the slider bar and heatmap corners. It saves these coordinates for future use.

```
python calibrate.py # Follow on-screen instructions # Coordinates saved to
config.json
```

Step 3: Configure Your Pairs

```
# In config.json: {   'pairs': ['XRPUSDT', 'BTCUSDT'],   'scan_interval_seconds':
3,   'threshold_k': 1.5,   'confidence_min': 0.60,   'log_dir': './logs' }
```

Step 4: Run the Learning Phase

First run the system in observation-only mode for 10-15 minutes per pair. It will watch and collect data without making predictions yet. This builds the initial threshold and feature baseline.

```
python main.py --mode observe --pair XRPUSDT --duration 15
```

Step 5: Start Live Predictions

```
python main.py --mode predict --pair XRPUSDT
```

The lime-colored overlay window will appear. Predictions show in the corner with direction, confidence, and target level.

Step 6: Review Logs and Improve

```
python analyze_logs.py --pair XRPUSDT # Shows: accuracy %, best performing
features, confusion matrix
```

---

9. Step-by-Step Build Guide for Mobile

Step 1: Enable ADB on Phone

17. Go to Settings > About Phone.
18. Tap Build Number 7 times to unlock Developer Options.
19. Go to Developer Options > Enable USB Debugging.
20. Connect phone to PC with USB cable.

21. Accept the USB debugging permission popup on phone.

Step 2: Verify ADB Connection

```
adb devices # Should show your device ID
```

Step 3: Configure for Mobile

```
# In config.json set: {   'platform': 'android',   'adb_device_id':
'your_device_id',   'screen_width': 720,   'screen_height': 1560,
'coinglass_app': true }
```

Step 4: Mobile Calibration

```
python calibrate_mobile.py # Takes screenshot via ADB, shows it on PC # Click on
slider, heatmap area to mark coordinates
```

Step 5: Run Mobile Controller

```
python main_mobile.py --mode predict --pair XRPUSDT # AI controls phone via ADB #
Predictions printed to PC terminal
```

Step 6: Optional - Show Predictions on Phone

Run a simple Flask web server on PC. Open http://your-pc-ip:5000 in phone browser to see live predictions on phone screen alongside the Coinglass app.

```
python prediction_server.py  # Start web server # Open on phone:
http://192.168.1.x:5000
```

---

10. File & Folder Structure

liquidity_ai/ main.py <- Entry point main_mobile.py <- Mobile ADB entry point calibrate.py <- Screen calibration calibrate_mobile.py <- Mobile calibration config.json <- Settings file modules/ screen_capture.py <- Screenshot & frame capture ocr_reader.py <- Read numbers from screen threshold_learner.py <- Auto-adjust slider logic liquidity_analyzer.py <- Find clusters and zones feature_extractor.py <- Build ML feature vectors online_model.py <- ML model (River library) prediction_engine.py <- Output final prediction overlay.py <- PyQt5 visual overlay log_manager.py <- Save and query logs adb_controller.py <- Android phone control logs/ xrpusdt_predictions.csv model_weights.pkl performance_stats.json analyze_logs.py <- Performance analysis tool prediction_server.py <- Flask server for mobile display

---

11. Model Performance & Improvement Over Time

The system measures its own accuracy. Here is how performance evolves:

Time Period	Expected Accuracy Range
First 10 minutes	Random ~50% (still learning baseline)
First hour	~55-60% (learning cluster patterns)
First day	~62-68% (pair behavior learned)
First week	~68-75% (time-of-day patterns captured)
First month	~72-80% (market regime patterns learned)

Note: accuracy above 60% on short-term crypto predictions is considered very good. The goal is consistent edge, not perfection.

Performance Dashboard

Run the analysis script anytime to see your model's health:

```
python analyze_logs.py  # Output example: # Pair: XRPUSDT | Total Predictions: 847 # UP Accuracy: 71% | DOWN Accuracy: 68% | Overall: 70% # Best Feature: liquidity_diff (importance: 0.34) # Worst Time: 03:00-04:00 UTC (accuracy drops to 52%) # Recommendation: Reduce confidence threshold to 0.65 for better coverage
```

12. Key Libraries Summary

Library	Purpose	Install Command
opencv-python	Image processing, color detection	pip install opencv-python
pytesseract	Read numbers from screen (OCR)	pip install pytesseract
pyautogui	Control mouse, click, drag	pip install pyautogui
mss	Fast screen capture	pip install mss
river	Online ML learning library	pip install river
scikit-learn	ML models and preprocessing	pip install scikit-learn
numpy	Math and array operations	pip install numpy
pandas	Log data management	pip install pandas
pyqt5	Visual overlay window	pip install pyqt5
flask	Web server for mobile display	pip install flask
adb-shell	Android phone control via Python	pip install adb-shell

13. Important Things to Know

Limitations

- The system reads a visual app (Coinglass), not raw data. If the app layout changes, recalibration is needed.
- OCR accuracy on small numbers is ~90-95%. Occasional misreads are expected.
- Mobile ADB approach is 300-500ms slower than PC due to USB/WiFi overhead.
- The model needs at least 30-50 predictions before it starts showing real accuracy improvements.

Best Practices

- Keep your screen resolution consistent. Do not resize the browser while the AI runs.
- Use a dedicated browser window for Coinglass, full-screened.
- Run the observation phase for at least 15 minutes before switching to prediction mode.
- Check logs weekly and retrain if accuracy drops below 55%.
- Test on paper first. Do not use AI predictions for real trades until you see consistent accuracy above 65% in your logs.

Legal & Ethical Note

This system is for educational and research purposes. Crypto trading involves significant financial risk. Always do your own due diligence. Past performance of ML predictions does not guarantee future results. Never risk money you cannot afford to lose.

14. Summary: What You Are Building

To wrap up everything in plain terms, here is what you are creating:

You are building a screen-reading, self-learning AI that acts like a highly focused trading analyst who never sleeps. It watches the Coinglass heatmap, knows exactly where big money is hiding at each price level, automatically adjusts its own tools for best visibility, and over time learns the unique way each crypto pair behaves around liquidity zones. It then predicts where price is heading next.

What You Are NOT Building	What You ARE Building
A bot that trades for you	A prediction and insight system
A get-rich-quick machine	A learning research tool

A system that is always right	A system that improves with every tick
Something that needs paid APIs	A screen-reading, free-data system

Good luck building. Every expert was once a beginner.

Start with the observation phase, watch your logs grow, and let the AI surprise you.